PESTO

A Full Stack Restaurant Web Application

TEAM MEMBERS:
2303A54038 - HASITHA VARSHINI
2303A54068 - VYSHNAVI
2303A54060 - SRIMUKHI
2303A54040 - SUSHMA

Introduction

What is Pesto?

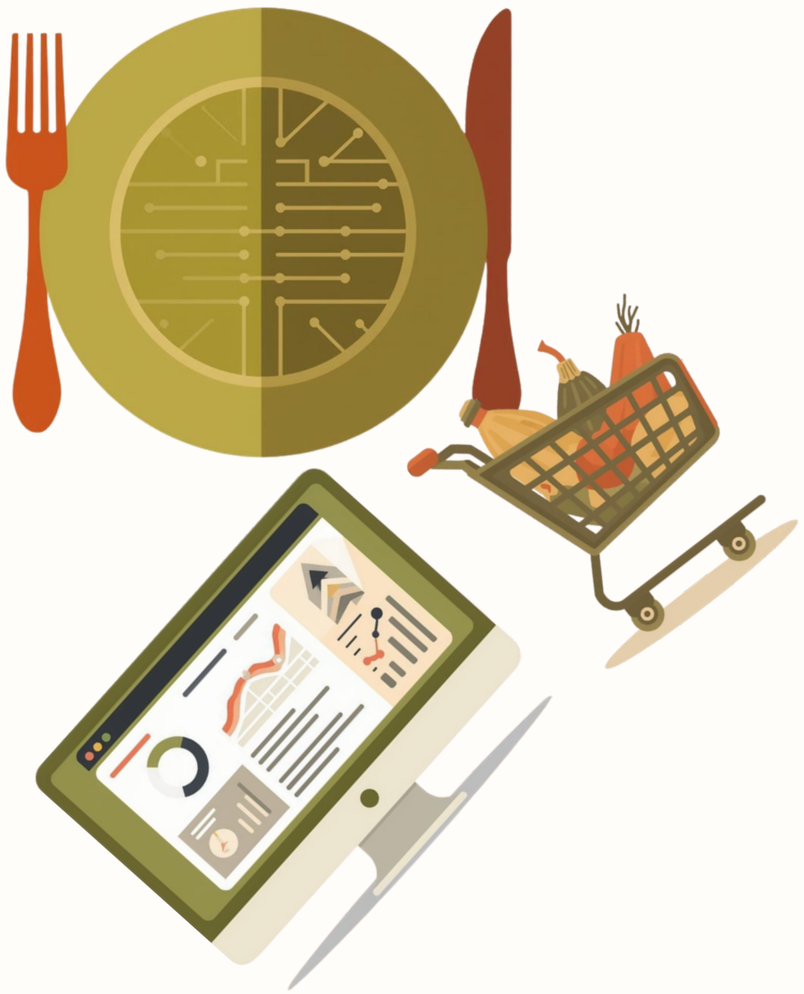
- A full-stack restaurant web application that digitizes the food ordering experience
- Built using React.js, Node.js, Express.js, and MySQL

The Problem it Solves

- Traditional restaurant ordering is manual, slow, and error-prone
- Pesto brings the entire ordering process online

Two Interfaces

- Customer — Browse menu, add to cart, and place orders
- Admin — View, accept, and manage incoming orders in real time



Objectives

- Build a dual-role web application serving both customers and restaurant admins
- Develop a responsive and user-friendly UI using React.js
- Implement secure user authentication for both customer and admin roles
- Design and develop RESTful APIs using Node.js and Express.js
- Create a well-structured and normalized MySQL database schema
- Enable real-time order placement and management between customer and admin
- Simulate a production-level full-stack project as part of academic learning

Stakeholders

Customers

End users who browse the menu, add items to cart, and place orders through the customer interface

Restaurant Admin

Manages the restaurant's operations by viewing, accepting, and managing incoming orders through the admin dashboard

Programming Languages, Tools & Software

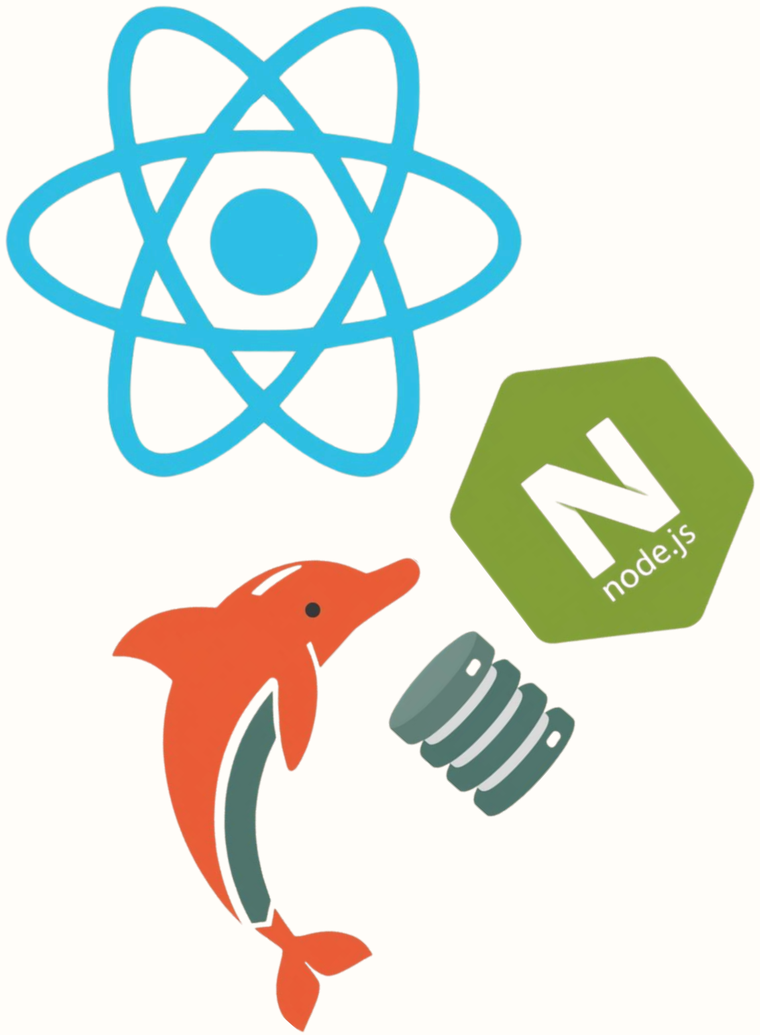
React.js — Frontend UI library

Node.js — Backend runtime

Express.js — Web framework

MySQL — Database

VS Code, GitHub



Prompts Used to Generate Code

Frontend (React.js)

- "I am building a restaurant web app called Pesto. Create a fully functional menu page in React.js where customers can browse food items with images, descriptions and prices, add them to a cart, update quantities, and proceed to checkout. Make it responsive and visually clean."

Backend (Node.js & Express.js)

- "I have a restaurant app called Pesto with two roles — customer and admin. Write a complete Node.js and Express.js backend with JWT-based authentication, routes for placing orders, fetching menu items, and an admin route to view and update order status in real time."

Database (MySQL)

- "Design a well-structured and normalized MySQL database schema for a restaurant web app called Pesto that handles user accounts, menu items with categories, order placement, order items, and order status tracking. Also write the important queries needed for the app to function."

PESTO

Workflow Flowchart-USER

Step 1
Register / Login

User visits the Pesto website and creates an account or logs in to access the platform

Step 2
Browse Menu

user explores available food items with images and prices

Step 3
Add to Cart & Checkout

user selects items, reviews the cart and places the order

Step 4
Status Update

Order status is updated in real time across the system. Customer receives confirmation and can track their order progress.

PESTO

Workflow Flowchart-ADMIN

Step 1
Admin Login

Restaurant admin logs in through the admin interface

Step 2
View Incoming Orders

Admin sees all newly placed orders on the dashboard

Step 3
Accept / Reject Order

Admin reviews and takes action on each order

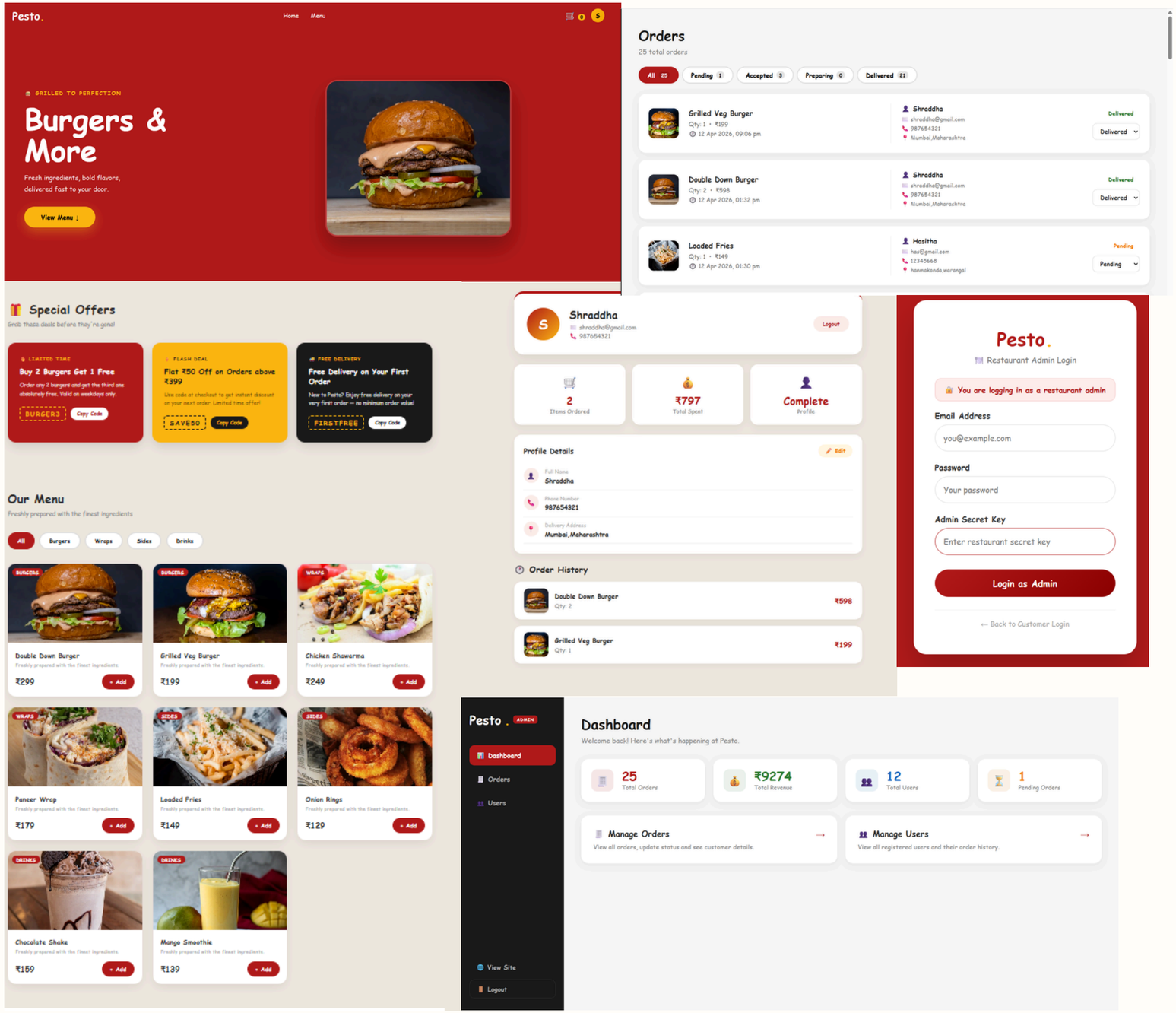
Step 4
Order Status Updated

Order status is updated in real time across the system. System reflects the updated status back to the customer.

PESTO

Frontend Views

Screenshots showcasing the customer-facing menu, cart checkout flow, and admin order management dashboard interfaces.



PESTO

Proof of Prompts Generating Code

Screenshots demonstrating how AI tools like ChatGPT and claude, gemini assisted in generating code for the Pesto restaurant application.

I am building a restaurant web app called Pesto using React.js. Create a fully functional menu page component where customers can browse food items with images, descriptions and prices, and add them to a cart. Make it responsive and visually clean."

Got it — you're building something real, so I'll give you a **clean, production-ready React menu page** (not just a toy example). This includes:

- Responsive layout (grid)
- Food cards with image, name, description, price
- Add to cart functionality
- Cart sidebar with totals
- Clean styling (no external libraries needed)

1. MenuPage.jsx

JavaScript

```javascript
      ) : (
        <>
          {cart.map((item) => (
            <div key={item.id} className="cart-item">
              <div>
                <h4>{item.name}</h4>
                <p>
                  ₹{item.price} × {item.quantity}
                </p>
              </div>
              <span>₹{item.price * item.quantity}</span>
            </div>
          ))}
          <hr />
          <h3>Total: ₹{getTotal()}</h3>
        </>
      )}
    </div>
  </div>
);
}
```

I have a restaurant app called Pesto with two roles — customer and admin. Write a complete Node.js and Express.js backend with JWT-based authentication, routes for placing orders, fetching menu items, and an admin route to view and update order status."

Reading the product self-knowledge skill to check for any relevant API details >

Now let me build the complete backend:

Created 15 files, ran 2 commands >

All tests pass. Now copy to output:

Copy to outputs >

Claude's response was interrupted Retry

give me manual code not files

Here's the complete code for all files:

Project Structure

```
pesto-backend/
├── server.js
├── .env
├── package.json
└── src/
    ├── app.js
    ├── config/db.js
    ├── middleware/auth.js
    ├── middleware/errorHandler.js
    ├── controllers/authController.js
    ├── controllers/menuController.js
    ├── controllers/orderController.js
    ├── routes/auth.js
    ├── routes/menu.js
    ├── routes/orders.js
    └── routes/admin.js
```

Setup

"Design a well-structured and normalized MySQL database schema for a restaurant web app called Pesto that handles user accounts, menu items with categories, order placement, order items, and order status tracking. Also write the important queries needed for the app to...

Building a database for a restaurant app like **Pesto** requires a balance between flexibility (for menu changes) and strict integrity (for financial transactions). Below is a normalized MySQL schema followed by the essential queries to keep the kitchen running smoothly.

1. Database Schema

We'll use a relational structure to ensure that if a dish price changes today, it doesn't retroactively alter the price of an order placed last week.

Table Definitions

SQL

```sql
CREATE DATABASE pesto_restaurant;
USE pesto_restaurant;

-- 1. User Accounts
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    address TEXT,
```

REFERENCES

Application

- GitHub Repository — https://github.com/hasitha1510/project

AI Tools

- ChatGPT — chat.openai.com
- Claude — claude.ai
- Gemini — gemini.google.com

Documentation

- React.js — reactjs.org
- Node.js — nodejs.org
- Express.js — expressjs.com
- MySQL — mysql.com